

Apex Auto Showroom

Car dealership website, dealer back office and Power BI analytics, as a demo app and a production version

IN ONE SENTENCE

A dealership's public showroom, lead pipeline and KPI dashboard, with Power BI exports, in one app.

BUILT WITH

React 19, TypeScript, Vite, Recharts; production: Node 22, Fastify 5, PostgreSQL, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	13

1. What this project is

Apex Auto is a complete car-dealership web application. The **public showroom** lets visitors search the stock, open a car page, try a finance calculator and send an enquiry or book a test drive. The **dealer back office** manages inventory, a six-stage lead pipeline and a KPI dashboard (units vs target, gross profit, lead funnel, response times, stock aging). A **Power BI starter kit** exports star-schema CSVs with 38 DAX measures.

The repository holds **two versions**. The root folder is a fast, fully client-side demo (data in the browser) that can be hosted free on GitHub Pages. The `claude-auto-showroom-production-grade` folder is the same product rebuilt for real use: a server with PostgreSQL, staff logins with roles, an audit trail, uploads and Docker deployment.

At a glance	
Type	Public website + staff back office + BI export
Users	Car buyers, salespeople, sales manager, dealer principal/admin
Stack	React 19, TypeScript, Vite, Recharts; production: Node 22, Fastify 5, PostgreSQL, Docker
Demo data	24 months: ~480 vehicles, ~430 sales, ~1,500 leads, 2 branches, 5 salespeople

Who uses it

User	What they do with it
Visitor / buyer	Searches stock with filters, opens car pages, calculates finance, sends enquiries and test-drive requests.
Salesperson	Works leads through the pipeline, records sales.
Sales manager	Manages inventory, watches response times and stock aging, reads the dashboard.
Owner / admin	Branches, targets, users, settings, Power BI exports, activity log.

Two versions in this repository

	Demo version (root folder)	Production version (....production-grade folder)
Data	Browser localStorage, one device	PostgreSQL shared by all staff
Admin access	Anyone who opens /admin	Logins with roles: admin, manager, sales
Cost and margin	Visible in the browser	Never sent to the public site; hidden from sales staff
Enquiries	Saved in the visitor's browser	Saved on the server; optional email alerts
Search	Downloads every car	Server-side filtering and paging
Photos	URL only	URL or checked upload (JPEG/PNG/WebP)
Power BI	CSV export	CSV export plus a read-only <code>bi</code> schema for scheduled refresh
Deployment	Static files (GitHub Pages)	Docker image, compose file, health checks

2. The business problem

Independent dealers often have a brochure website that isn't connected to their stock, leads kept in email and notebooks, and no reliable view of profit per car, stock age or how fast salespeople answer enquiries. Cars sit too long, leads go cold, and the owner finds out at month end.

The client wants a **modern showroom website connected to a simple back office**, and **dashboards** (in the app and in Power BI) that show the numbers dealers actually manage by.

Pain point today	What it costs the business	How this project solves it
Website not connected to stock	Sold cars still advertised; no online leads	Showroom reads the live inventory; enquiries flow into the pipeline
Leads tracked in email	Slow replies, lost sales	Six-stage lead board; first-response time recorded; unanswered leads flagged
No view of stock age	Cars lose value while sitting	Stock-aging chart and oldest-stock table; over-90-days KPI
Profit per car unknown	Pricing by gut feel	Gross per unit, margin and salesperson leaderboard
Reporting by hand	Owner sees problems late	Live KPI dashboard and Power BI kit with 38 measures

Value to the business

- More enquiries from the website, answered faster.
- Stock turned faster through aging visibility.
- One source of truth for sales performance, in the app and in Power BI.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Showroom	Inventory with filters (make, body, fuel, gearbox, condition, branch, price, year, mileage), search and sort; filters in the URL so searches can be shared.
FR-02	Showroom	Vehicle pages with specs, features, similar cars and a finance calculator (deposit, term, APR).
FR-03	Showroom	Enquiry, test-drive and finance-quote forms with validation; each submission becomes a lead.
FR-04	Showroom	Responsive down to phone width with automatic dark mode.
FR-05	Back office	Dashboard filtered by period and branch: units vs target, revenue, gross profit and margin, gross per unit, days to sell, lead conversion, first-response time, unanswered leads, stock value, stock over 90 days; charts with table views.
FR-06	Back office	Inventory: add/edit vehicles with validation, reserve, record a sale (price, date, salesperson, gross preview), delete.
FR-07	Back office	Leads: six-stage board; stage changes logged; first move out of New records response time; unanswered leads flagged.
FR-08	Data & BI	Download star-schema CSVs for Power BI; 38 DAX measures, theme and modelling guide; backup/restore (demo).
FR-09	Production additions	Staff logins with roles; cost and margin never sent to the public site and hidden from sales staff.
FR-10	Production additions	Server-side search and paging; photo uploads checked by content; lead spam protection (rate limits, honeypot); email alerts; sitemap.
FR-11	Production additions	Audit log; read-only bi database schema for scheduled Power BI refresh.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Security (production)	Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.
NFR-02	Security (production)	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-03	Security (production)	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-04	Security (production)	An activity (audit) log of logins and every business action, viewable by admins.
NFR-05	Security (production)	No customer personal data in Power BI exports or bi views.
NFR-06	Integrity (production)	Sold cars can't be deleted; only an admin can reverse a sale; a car can't be sold twice; two editors get a conflict warning.

ID	Quality	Requirement
NFR-07	Consistency	Database search returns the same cars in the same order as the shared browser logic; dashboard numbers equal the shared analytics over the whole database.
NFR-08	Usability	Phone layout, dark mode, every chart has a table view.
NFR-09	Operations (production)	Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances.
NFR-10	Operations (production)	Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.

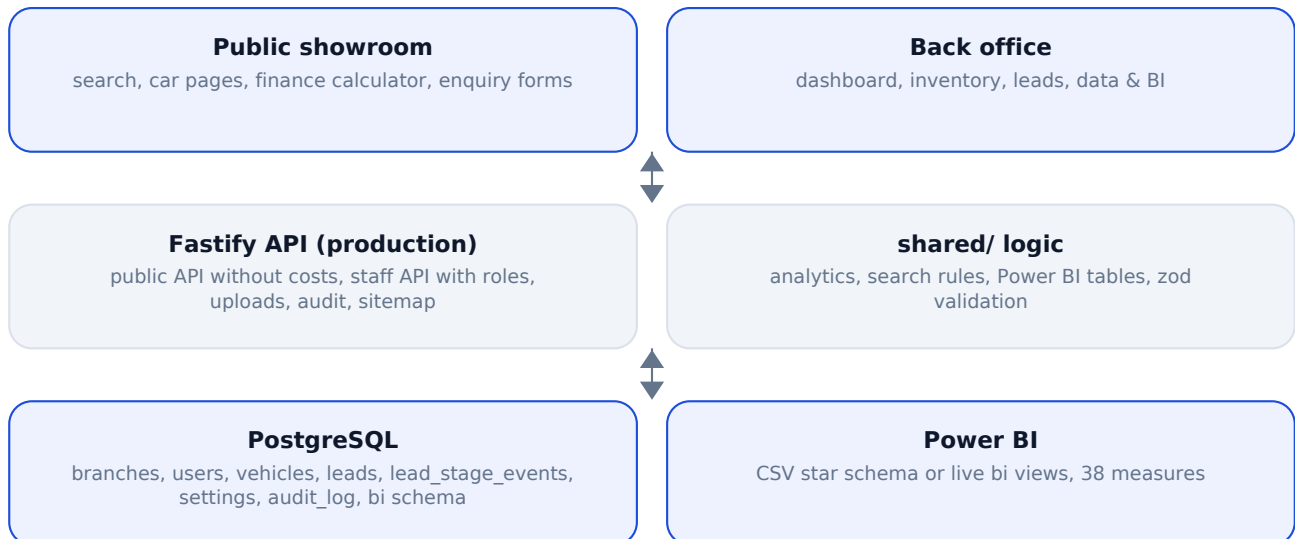
Out of scope for this release

- Online payments and reservations with deposits.
- DMS integrations (CDK, Reynolds, Dealertrack) and marketplace feeds.
- Server-side rendering of car pages (social previews show the generic title).

4. How the solution is designed

The demo keeps all logic in pure, unit-tested TypeScript modules (`src/lib`) and state in a store persisted to `localStorage`. The production version moves that logic into `shared/` so the server and browser compute the same numbers, and puts the data in PostgreSQL behind a Fastify API.

Architecture



Technology choices

Part	Choice	Why
Web	React 19, TypeScript, Vite, React Router, Recharts	Fast SPA; hash routing lets the demo run on any static host.
Demo data	Seeded generator (<code>src/data</code>)	24 months of realistic history ending today, reproducible.
Server (prod)	Node 22 + Fastify 5 + PostgreSQL	Shared TypeScript logic; transactions for sales; constraints for integrity.
BI	CSV star schema + DAX measures	Works with Power BI or Excel; <code>bi</code> views for scheduled refresh.
Tests	Vitest, Playwright	Unit tests for all maths; browser smoke tests of every page.

Data model

Table / file	What it holds
vehicles	Stock: make, model, year, mileage, price, cost, status, branch, photos, dates in stock and sold, sale details.
leads / lead_stage_events	Enquiries with type, source, vehicle, salesperson, stage; every stage change with time (response time).
branches / users / sessions	Showrooms, staff with roles, sessions (production).
settings / audit_log	Branding, targets and defaults; activity log (production).
Power BI CSVs	FactSales, FactLeads, FactLeadStageHistory, FactTargets, DimVehicle, DimDate, DimBranch, DimSalesperson (star schema).

Key business rules

- **Gross profit** = sale price – vehicle cost; **days to sell** = sale date – stock date.
- **Response time** = time from lead creation to its first move out of New.
- **Finance calculator** uses the standard amortisation formula on price – deposit, APR and term.
- **Stock aging buckets:** 0-30, 31-60, 61-90, 90+ days.

Folder layout

```
Root (demo):  src/config.ts · types.ts · data/ (generator) · lib/ (analytics, inventory,
               finance, powerbi)
               store/ (state + localStorage) · pages/ · powerbi/ (CSVs, measures.dax, theme) ·
               scripts/
claude-auto-showroom-production-grade/:
               shared/ · server/ (Fastify, migrations, tests) · web/ · powerbi/ · docs/ ·
               Dockerfile
```

5. Build it from scratch, step by step

Part A builds the demo (steps 1–7): the fastest way to agree the product with the client. Part B (steps 8–14) turns it into the production system without throwing the demo's logic away.

Before you start

- Node.js 22+, Git; PostgreSQL 14+ and Docker for Part B.
- React and TypeScript basics; SQL for Part B; Power BI Desktop to use the BI kit.

Step 1. Part A · Scaffold the demo app

```
npm create vite@latest claude-auto-showroom -- --template react-ts
cd claude-auto-showroom && npm i react-router-dom recharts && npm i -D vitest playwright-core
```

1.1 Add `src/config.ts` for branding (name, currency, locale, branches, targets, salespeople, finance defaults) and `src/types.ts` for Vehicle, Lead, Sale.

Done when: `npm run dev` shows an empty app with routing (`/` and `/#/admin`).

Step 2. Part A · Demo data generator

2.1 Write a seeded random generator and a model catalogue; generate 24 months of stock, sales and leads ending today with seasonality, aging stock and some slow lead responses.

2.2 Unit-test the data's consistency (no car sold twice, sale after stock date).

Done when: The generator produces the same data for the same seed.

Step 3. Part A · Business logic as pure functions

3.1 `lib/inventory.ts` (filters, search, sort), `lib/finance.ts` (monthly payment), `lib/analytics.ts` (KPIs, funnel, aging, leaderboard), `lib/powerbi.ts` (star-schema tables) — each with unit tests.

Done when: `npm test` passes for all lib modules.

Step 4. Part A · Store and persistence

4.1 A React context store with actions (add/edit vehicle, reserve, record sale, create lead, move stage) persisted to `localStorage`, with backup/restore/reset.

Done when: Changes survive a page reload.

Step 5. Part A · Showroom pages

5.1 Home and inventory with URL filters, vehicle page with gallery placeholder, specs, similar cars, finance calculator and validated enquiry/test-drive/finance forms that create leads.

Done when: An enquiry sent on the site appears in the admin lead board.

Step 6. Part A - Back office

6.1 Dashboard with period/branch filters, KPI cards and Recharts charts (each with a table view); inventory table and forms; lead board; Data & Power BI page with CSV downloads.

Done when: All dashboard numbers change with the filters and match the analytics tests.

Step 7. Part A - Power BI kit, tests and publishing

7.1 `npm run export:powerbi` writes the CSVs; write `measures.dax`, a theme and a modelling guide (relationships, date table, measures).

7.2 Playwright smoke test clicking through every page, at phone width and in dark mode.

7.3 GitHub Actions deploy workflow to GitHub Pages (Settings → Pages → Source: GitHub Actions).

Done when: Tests pass and the site is live on Pages.

Step 8. Part B - Set up the workspace

One repository with a server, a web app and shared code used by both.

8.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir claudé-auto-showroom-production-grade && cd claudé-auto-showroom-production-grade && git
init
npm init -y          # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod @fastify/multipart
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

8.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 9. Part B - Configuration, database connection and migrations

9.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`, `SMTP_URL`) and refuse to start with a clear message when one is wrong.

9.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.

9.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 10. Part B · Schema and shared logic

- 10.1 Move `analytics`, search rules, Power BI tables and validation into `shared/` so both server and browser use them.
- 10.2 Tables: branches, users, sessions, vehicles (with version), leads, lead_stage_events, settings, audit_log; plus a `bi` schema of read-only views without personal data.

Done when: Migrations apply; shared tests pass.

Step 11. Part B · Logins, roles and request security

The same proven foundation every production app here uses.

- 11.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 11.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 11.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 11.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 11.5 **Roles:** admin (users, settings, branches, reverse a sale), manager (stock, costs, reports) and sales (leads, sales; never sees cost). The public API is a separate set of routes that never returns cost, sale price or staff details. A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 11.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 11.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 12. Part B · API and web app

- 12.1 Public routes: search with server-side filtering and paging (visitor text matched literally), vehicle page, lead form with rate limit and honeypot, sitemap.
- 12.2 Staff routes: inventory with optimistic locking, record/reverse sale in a transaction, lead stages, dashboard computed with the shared analytics, photo upload checked by content, CSV exports, audit log.
- 12.3 Point the React app at the API (TanStack Query), add login and role-aware navigation.

Done when: API tests: public responses contain no cost; a car can't be sold twice; stale edits get a conflict.

Step 13. Part B · Test it end to end

- 13.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 13.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 13.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then run a visitor's enquiry, a salesperson working it, a manager recording a sale, and the dashboard, on desktop and phone. Fail on any script error, and save screenshots.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 14. Part B · Package, automate and document

- 14.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 14.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 14.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 14.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **Demo:** unit tests for analytics, inventory search, Power BI tables and validation; Playwright smoke test of every page including phone width and dark mode.
- **Production:** database search equals the shared browser logic; dashboard equals shared analytics over the whole database; access control (public never sees cost, sales never sees cost, CSRF, cross-site posts refused); lockout; sold-once and conflict rules; uploads checked by bytes; lead spam protection; full-stack browser smoke test.

```
# demo
npm test && npm run build && npm run smoke
# production (inside the -production-grade folder)
npm run typecheck && npm test && npm run build && npm run smoke
```

Deploying and running it

- **Demo:** GitHub Pages (Settings → Pages → Source: GitHub Actions) or any static host serving `dist/`.

```
cd claude-auto-showroom-production-grade
cp .env.example .env # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email owner@yourdealer.com --name "Owner Name"
# demo data: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js
```

Known limits

- Demo: data lives in one browser and `/admin` has no login — for demonstrations only.
- Production: rate limits per server instance; photos stored as uploaded (no resizing); no password-reset emails; car pages rendered in the browser.

Sensible next steps

- Replace this showroom's stock with the dealer management system's API.
- Email/WhatsApp lead alerts; photo resizing and a CDN.
- Publish stock to marketplaces via the dealership-automation project.